# LAPORAN PRAKTIKUM 1

Struktur Data

RAFKI AHMAD PAGAMANDA

2311533016

Kelas D

Pengenalan Class dan Object dalam Pemrograman Java Menggunakan Eclipse

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

# Pengenalan Class dan Object dalam Pemrograman Java Menggunakan Eclipse

## A. Tujuan

Tujuan dari praktikum ini adalah utk memahami konsep dasar dalam pemrograman berorientasi objek, khususnya terkait penggunaan class dan object. Pada praktikum ini saya mempelajari cara membuat class sederhana seperti Jam dan Mobil beserta atribut dan method yg terdapat di dalamnya. Selain itu, praktikum ini jg membantu saya memahami bagaimana cara menjalankan program menggunakan class driver sebagai bagian utama dari program. Dgn menggunakan Eclipse IDE, saya dapat mengimplementasikan serta menguji program yg telah dibuat sehingga lebih memahami alur dasar pembuatan program berbasis object.

## B. Teori Singkat

Pada praktikum ini, saya mulai memahami konsep dasar dari pemrograman berorientasi objek atau biasa disebut OOP. Secara umum, OOP itu adalah cara membuat program yg berfokus pada object, jadi bukan cuma sekadar kumpulan perintah saja, tapi lebih terstruktur.

Di dalam OOP, ada beberapa hal penting yg saya pelajari, seperti class dan object. Class bisa dibilang sebagai rancangan awal, yg nantinya akan digunakan utk membuat object. Di dalam class tersebut, saya bisa menentukan atribut (data) dan method (fungsi) sesuai kebutuhan. Sedangkan object merupakan hasil dari class itu sendiri, jadi bisa digunakan langsung dalam program.

Selain itu, saya jg belajar bagaimana cara menjalankan program menggunakan class driver. Biasanya di dalamnya terdapat method main yg berfungsi sebagai titik awal program dijalankan. Dari sini, saya bisa menghubungkan class yg sudah dibuat sebelumnya agar bisa diuji dan dilihat hasilnya. Dgn adanya praktikum ini, saya jadi lebih paham gambaran dasar bagaimana sebuah program OOP dibentuk, walaupun masih dalam bentuk yg sederhana.

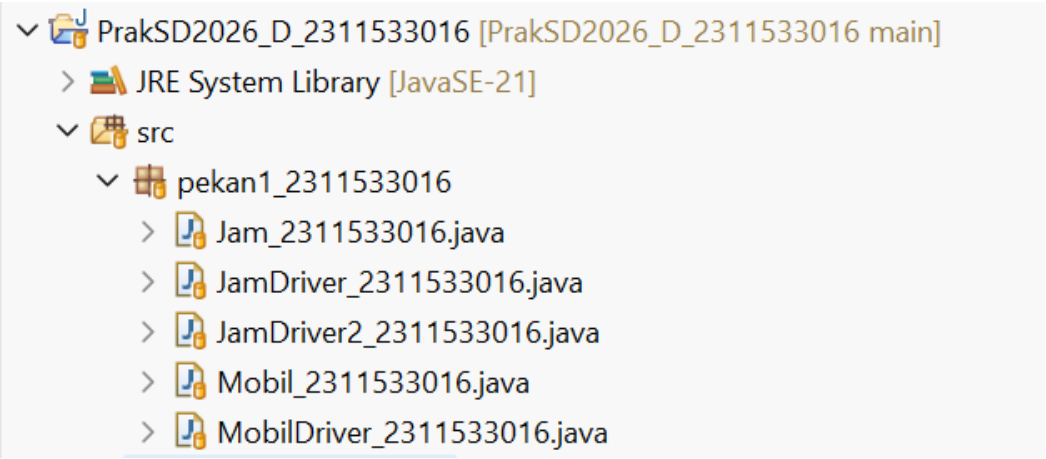
## C. Alat dan Bahan

Pada praktikum ini, saya menggunakan beberapa alat dan bahan yg cukup sederhana. Perangkat utama yg digunakan adalah laptop atau komputer yg sudah terinstall aplikasi Eclipse IDE sebagai media utk menulis dan menjalankan program. Selain itu, saya jg menggunakan bahasa pemrograman Java karena pada praktikum ini berfokus pada konsep OOP.

Tidak ada bahan khusus yg digunakan selain source code yg dibuat selama praktikum, seperti class Jam, Mobil, dan class driver yg digunakan utk menjalankan program. Semua proses dilakukan langsung di dalam IDE, mulai dari penulisan kode sampai pengujian program.

# D. Langkah-langkah

1. Pembuatan Package dan Class




Pd langkah pertama ini, sy membuat sebuah package baru di dalam project yg ada di Eclipse IDE. Package tersebut diberi nama pekan1_2311533016, yg digunakan sbg penanda praktikum pekan 1 sekaligus identitas sy. Setelah itu, sy mulai membuat beberapa class di dalam package tersebut, yaitu Jam, JamDriver, JamDriver2, Mobil, dan MobilDriver. Pembuatan class ini dilakukan secara bertahap sesuai kebutuhan praktikum. Class Jam dan Mobil berfungsi sbg class utama yg berisi atribut dan method, sedangkan class driver seperti JamDriver, JamDriver2, dan MobilDriver digunakan utk menjalankan program.

2. Query 2

```java
package pekan1_2311533016;

public final class Jam_2311533016 {

    private int hh; // 0..23
    private int mm; // 0..59
    private int ss; // 0..59

    // ---- Validator ----
    public static boolean isValid(int h, int m, int s) {
        return (0 <= h && h <= 23) &&
               (0 <= m && m <= 59) &&
               (0 <= s && s <= 59);
    }

    // ---- Konstruktor / Kreator ----
    public Jam_2311533016(int h, int m, int s) {
        this.hh = h;
        this.mm = m;
        this.ss = s;
    }

    // ---- Selector (Getter) ----
    public int getHour() {
        return hh;
    }

    public int getMinute() {
        return mm;
    }

    public int getSecond() {
        return ss;
    }

    // ---- Mutator (Setter) ----
    public void setHour(int h) {
        this.hh = h;
    }

    public void setMinute(int m) {
        this.mm = m;
    }

    public void setSecond(int s) {
        this.ss = s;
    }

    // ---- Konversi ----
    public int toSeconds() {
        return hh * 3600 + mm * 60 + ss;
```

```java
    // ---- Konversi ----
    public int toSeconds() {
        return hh * 3600 + mm * 60 + ss;
    }

    public static Jam_2311533016 fromSeconds(int total) {
        if (total < 0) throw new IllegalArgumentException("detik negatif");

        total %= 24 * 3600; // wrap 24 jam

        int h = total / 3600;
        total %= 3600;

        int m = total / 60;
        int s = total % 60;

        return new Jam_2311533016(h, m, s);
    }

    // ---- Perbandingan ----
    public int compareTo(Jam_2311533016 other) {
        return Integer.compare(this.toSeconds(), other.toSeconds());
    }

    public boolean equals(Object o) {
        if (!(o instanceof Jam_2311533016)) return false;
        Jam_2311533016 j = (Jam_2311533016) o;
        return hh == j.hh && mm == j.mm && ss == j.ss;
    }

    public int hashCode() {
        return java.util.Objects.hash(hh, mm, ss);
    }

    // ---- Aritmatika ----
    public Jam_2311533016 plus(Jam_2311533016 other) {
        return fromSeconds(this.toSeconds() + other.toSeconds());
    }

    public Jam_2311533016 minus(Jam_2311533016 other) {
        return fromSeconds(Math.floorMod(this.toSeconds() - other.toSeconds(), 24 * 3600));
    }

    public Jam_2311533016 nextSecond() {
        return fromSeconds(this.toSeconds() + 1);
    }

    public Jam_2311533016 nextNSeconds(int n) {
        return fromSeconds(this.toSeconds() + Math.max(0, n));
    }
```

```java
    public Jam_2311533016 prevSecond() {
        return fromSeconds(Math.floorMod(this.toSeconds() - 1, 24 * 3600));
    }

    public Jam_2311533016 prevNSeconds(int n) {
        return fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n), 24 * 3600));
    }

    // ---- Durasi ----
    public static int durasiDetik(Jam_2311533016 jAw, Jam_2311533016 jAkh) {
        return jAkh.toSeconds() - jAw.toSeconds();
    }

    // ---- toString ----
    public String toString() {
        return String.format("%02d:%02d:%02d", hh, mm, ss);
    }
}
```

Pd langkah ini, sy membuat class Jam_2311533016 yg digunakan utk merepresentasikan waktu dalam bentuk jam, menit, dan detik. Di dalam class ini terdapat atribut hh, mm, dan ss yg bertipe integer, dimana masing-masing menyimpan nilai jam (0–23), menit (0–59), dan detik (0–59). Selain itu, terdapat method isValid yg berfungsi utk mengecek apakah nilai waktu yg dimasukkan sudah sesuai dgn batas yg ditentukan atau belum. Sy jg membuat konstruktor utk menginisialisasi nilai awal dari object jam, serta beberapa method getter dan setter yg digunakan utk mengambil dan mengubah nilai atribut. Selain bagian dasar tersebut, class ini jg memiliki beberapa fungsi tambahan yg cukup penting. Terdapat method toSeconds dan fromSeconds yg digunakan utk mengkonversi waktu ke dalam bentuk detik dan sebaliknya. Kemudian ada method perbandingan seperti compareTo, equals, dan hashCode yg digunakan utk membandingkan dua object jam. Sy jg menambahkan method aritmatika seperti plus, minus, serta fungsi penambahan dan pengurangan detik seperti nextSecond dan prevSecond. Di bagian akhir, terdapat method toString yg digunakan utk menampilkan waktu dalam format jam:menit:detik, sehingga output yg dihasilkan menjadi lebih rapi dan mudah dibaca.

3. Pembuatan Class JamDriver

```java
package pekan1_2311533016;

public class JamDriver_2311533016 {
    public static void main(String[] args) {

        Jam_2311533016 a = new Jam_2311533016(23, 59, 50);
        Jam_2311533016 b = new Jam_2311533016(0, 0, 15);

        System.out.println("a = " + a);
        System.out.println("b = " + b);
        System.out.println("a+b = " + a.plus(b));
        System.out.println("next 20s = " + a.nextNSeconds(20));
        System.out.println("durasi(a,b) = " + Jam_2311533016.durasiDetik(a, b));
        System.out.println("a.compareTo(b) = " + a.compareTo(b));
    }
}
```

Pd langkah ini, sy membuat class JamDriver_2311533016 yg berfungsi sbg class utama utk menjalankan program. Di dalam class ini terdapat method main yg digunakan sbg titik awal eksekusi program. Pada bagian awal, sy membuat dua object dari class Jam, yaitu variabel a dan b, dgn nilai waktu yg berbeda, dimana a berisi waktu 23:59:50 dan b berisi waktu 00:00:15. Selanjutnya, sy melakukan beberapa percobaan utk menguji method yg ada pd class Jam. Hasil dari object ditampilkan menggunakan System.out.println, seperti menampilkan nilai waktu, hasil penjumlahan waktu (plus), penambahan detik (nextNSeconds), serta menghitung durasi antara dua waktu menggunakan method durasiDetik. Selain itu, sy jg mencoba method compareTo utk melihat perbandingan antara dua object jam.

4. Pembuatan Class JamDriver2

```java
package pekan1_2311533016;

import java.util.Scanner;

public class JamDriver2_2311533016 {
    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        System.out.println("=== Program Driver Objek Jam ===");

        // 1. Input Jam Pertama
        System.out.println("\n--- Input Jam 1 ---");
        Jam_2311533016 j1 = buatJamDariInput(input);

        // 2. Input Jam Kedua
        System.out.println("\n--- Input Jam 2 ---");
        Jam_2311533016 j2 = buatJamDariInput(input);

        // 3. Menampilkan Data
        System.out.println("\n--- Hasil Operasi ---");
        System.out.println("Jam 1 (String)      : " + j1.toString());
        System.out.println("Jam 2 (String)      : " + j2.toString());
        System.out.println("Jam 1 dalam detik   : " + j1.toSeconds());
        System.out.println("Jam 2 dalam detik   : " + j2.toSeconds());

        // 4. Operasi Relasional (Perbandingan)
        int perbandingan = j1.compareTo(j2);

        if (perbandingan > 0) {
            System.out.println("Status : Jam 1 lebih lambat (setelah) Jam 2");
        } else if (perbandingan < 0) {
            System.out.println("Status : Jam 1 lebih awal (sebelum) Jam 2");
        } else {
            System.out.println("Status : Jam 1 dan Jam 2 sama persis");
        }

        // 5. Operasi Aritmatika
        System.out.println("Durasi (J1 ke J2) : " + Jam_2311533016.durasiDetik(j1, j2) + " detik");

        Jam_2311533016 jNext = j1.nextSecond();
        System.out.println("Jam 1 Detik Berikutnya: " + jNext);

        Jam_2311533016 jPrev = j1.prevSecond();
        System.out.println("Jam 1 Detik Sebelumnya: " + jPrev);

        // 6. Operasi Penjumlahan Jam
        Jam_2311533016 jHasilPlus = j1.plus(j2);
        System.out.println("Hasil J1 + J2 : " + jHasilPlus);

        input.close();
```

```java
        System.out.println("\nProgram Selesai.");
    }

    // ☑ Method versi FINAL (dari slide terakhir - pakai validasi)
    private static Jam_2311533016 buatJamDariInput(Scanner sc) {
        int h, m, s;

        while (true) {
            System.out.print("Masukkan Jam (0-23)   : ");
            h = sc.nextInt();

            System.out.print("Masukkan Menit (0-59) : ");
            m = sc.nextInt();

            System.out.print("Masukkan Detik (0-59) : ");
            s = sc.nextInt();

            // validasi
            if (Jam_2311533016.isValid(h, m, s)) {
                return new Jam_2311533016(h, m, s);
            } else {
                System.out.println("[Error] Input tidak valid! Silakan ulangi.\n");
            }
        }
    }
}
```

Pd langkah ini, sy membuat class JamDriver2_2311533016 yg berfungsi utk menjalankan program dgn input langsung dari user. Berbeda dgn driver sebelumnya yg menggunakan data tetap, pd class ini sy memanfaatkan Scanner utk menerima input jam, menit, dan detik dari user. Proses input dilakukan sebanyak dua kali utk membuat dua object Jam, yaitu j1 dan j2, dimana input tersebut akan divalidasi terlebih dahulu menggunakan method isValid agar sesuai dgn rentang waktu yg benar. Setelah data berhasil diinput, program kemudian menampilkan berbagai hasil operasi yg telah disediakan pd class Jam. Mulai dari menampilkan waktu dalam bentuk string dan detik, melakukan perbandingan menggunakan compareTo, hingga operasi aritmatika seperti menghitung durasi, detik berikutnya (nextSecond), detik sebelumnya (prevSecond), serta penjumlahan dua waktu (plus). Selain itu, terdapat method tambahan buatJamDariInput yg digunakan utk mempermudah proses input sekaligus memastikan data yg dimasukkan valid.

5. Pembuatan Class Mobil

```java
package pekan1_2311533016;

public class Mobil_2311533016 {

    private String nama;
    private int tahun;
    private int cc;
    private double harga;
    private String merk;

    // Konstruktor
    public Mobil_2311533016(String nama, int tahun, int cc, double harga, String merk) {
        this.nama = nama;
        this.tahun = tahun;
        this.cc = cc;
        this.harga = harga;
        this.merk = merk;
    }

    // Selector (Getter)
    public String getNama() {
        return nama;
    }

    public int getTahun() {
        return tahun;
    }

    public int getCc() {
        return cc;
    }

    public double getHarga() {
        return harga;
    }

    public String getMerk() {
        return merk;
    }

    // Mutator (Setter)
    public void setNama(String nama) {
        this.nama = nama;
    }

    public void setTahun(int tahun) {
        this.tahun = tahun;
    }

    public void setCc(int cc) {
```

```java
        this.cc = cc;
    }

    public void setHarga(double harga) {
        this.harga = harga;
    }

    public void setMerk(String merk) {
        this.merk = merk;
    }

    // Method tambah mobil (simulasi)
    public void tambahMobil() {
        System.out.println("Mobil berhasil ditambahkan!");
    }

    // Method hapus mobil (simulasi)
    public void hapusMobil() {
        System.out.println("Mobil berhasil dihapus!");
    }

    // toString
    public String toString() {
        return nama + " | " + tahun + " | " + cc + "cc | Rp" + harga + " | " + merk;
    }
}
```

Pd langkah ini, sy membuat class Mobil_2311533016 yg digunakan utk merepresentasikan data sebuah mobil. Di dalam class ini terdapat beberapa atribut seperti nama, tahun, cc, harga, dan merk, yg masing-masing menyimpan informasi terkait mobil tersebut. Sy jg membuat konstruktor yg digunakan utk mengisi nilai awal saat object dibuat, sehingga setiap object mobil bisa langsung memiliki data lengkap sejak awal.

Selain itu, sy menambahkan method getter dan setter utk mengambil serta mengubah nilai dari atribut yg ada. Hal ini memudahkan dalam pengelolaan data tanpa harus mengakses atribut secara langsung. Terdapat jg method tambahan seperti tambahMobil dan hapusMobil yg bersifat simulasi utk menampilkan aksi terhadap data mobil. Di bagian akhir, sy menggunakan method toString utk menampilkan seluruh informasi mobil dalam satu baris agar lebih mudah dibaca saat program dijalankan.

6. Pembuatan Class MobilDriver

```java
package pekan1_2311533016;

public class MobilDriver_2311533016 {
    public static void main(String[] args) {

        // Membuat objek mobil
        Mobil_2311533016 m1 = new Mobil_2311533016("Avanza", 2020, 1300, 150000000, "Toyota");

        // Menampilkan data
        System.out.println("Data Mobil:");
        System.out.println(m1);

        // Testing method
        m1.tambahMobil();
        m1.hapusMobil();

        // Testing mutator
        m1.setHarga(160000000);
        System.out.println("Harga setelah update:");
        System.out.println(m1);
    }
}
```

Pd langkah ini, sy membuat class MobilDriver_2311533016 utk menjalankan dan menguji class Mobil. Sy membuat object m1 dgn beberapa data, lalu menampilkannya menggunakan toString. Setelah itu, sy mencoba beberapa method seperti tambahMobil, hapusMobil, serta setHarga utk mengubah data, kemudian menampilkan kembali hasilnya

## E. Kesimpulan

Dari praktikum yg telah saya lakukan, saya jadi lebih memahami konsep dasar OOP, terutama terkait penggunaan class dan object. Sy belajar bagaimana membuat class seperti Jam dan Mobil beserta atribut dan method yg ada di dalamnya, serta bagaimana menjalankannya menggunakan class driver. Selain itu, saya jg memahami penggunaan getter, setter, konstruktor, dan beberapa method tambahan utk operasi data. Dgn adanya praktikum ini, sy menjadi lebih paham alur dasar pembuatan program berbasis object walaupun masih dalam bentuk sederhana.